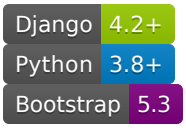
🚀 Enhanced Django Task Manager

A comprehensive task management system with **role-based access control**, approval workflows, and modern Bootstrap 5 UI. Built with Django 4.2+ and designed for small to medium teams.



✨ Features

🔐 Role-Based Access Control

- **Admin Users**: Full system management, task assignment, employee oversight
- **Employee Users**: Personal task management, status updates, completion workflow

📋 Task Management

- ✅ Task assignment with priorities and categories
- ✅ Due date tracking with overdue alerts
- ✅ File attachments support
- ✅ Status tracking (Assigned → In Progress → Completed → Approved)
- ✅ Approval workflow system

🎨 Modern UI/UX

- ✅ Responsive Bootstrap 5 design
- ✅ Role-specific dashboards
- ✅ Real-time notifications
- ✅ Mobile-friendly interface

📊 Advanced Features

- ✅ Performance tracking per employee
- ✅ Task filtering and search
- ✅ Comments and history tracking
- ✅ Comprehensive admin panel

🚀 Quick Start Guide

Prerequisites

- Python 3.8 or higher
- Git (optional)

Step 1: Create Virtual Environment

```
# Create virtual environment
python -m venv venv
```

Step 2: Activate Virtual Environment

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Setup Database & Initial Data

```
python setup_enhanced.py
```

This will:

- Create database migrations
- Apply migrations
- Create default priorities and categories
- Set up user profiles

Step 5: Create Demo Users

```
python create_test_users.py
```

This creates demo accounts for testing:

- **Admin**: `admin` / `admin123`
- **Employees**: `john.doe`, `jane.smith`, etc. / `emp123`

Step 6: Start Development Server

```
python manage.py runserver
```

 Success! Open your browser to: http://127.0.0.1:8000/

Demo Login Credentials

Admin Account

- **Username**: `admin`
- **Password**: `admin123`
- **Access**: Full system management

Employee Accounts

- **Username**: `john.doe`, `jane.smith`, `mike.wilson`, `sarah.johnson`
- **Password**: `emp123`
- **Access**: Personal task management

Usage Workflow

For Administrators:

1. **Login** → Access admin dashboard with complete overview
2. **Assign Tasks** → Create tasks and assign to employees
3. **Monitor Progress** → Track employee performance and task status
4. **Review Completions** → Approve or reject completed tasks
5. **Manage System** → Access admin panel for advanced management

For Employees:

1. **Login** → View personal dashboard with assigned tasks
2. **Start Work** → Update task status to "In Progress"
3. **Complete Tasks** → Mark tasks complete with notes
4. **Track Performance** → View personal statistics and history

📁 Project Structure

```
enhanced-task-manager/
├── 📄 manage.py               # Django management script
├── 📄 requirements.txt        # Project dependencies
├── 📄 setup_enhanced.py       # Setup automation script
├── 📄 create_test_users.py    # Demo users creation
├── 📄 README.md              # This file
│
├── 📁 task_manager/           # Main Django project
│   ├── 📄 settings.py         # Project configuration
│   ├── 📄 urls.py             # URL routing
│   └── 📄 wsgi.py            # WSGI configuration
│
├── 📁 tasks/                  # Main application
│   ├── 📄 models.py           # Database models
│   ├── 📄 views.py            # Application logic
│   ├── 📄 forms.py            # Form handling
│   ├── 📄 admin.py            # Admin interface
│   └── 📄 urls.py             # App URL patterns
│
├── 📁 templates/              # HTML templates
│   ├── 📄 base.html           # Base template
│   └── 📁 tasks/              # Task-specific templates
│
├── 📁 static/                 # Static files
│   ├── 📁 css/style.css       # Custom styling
│   └── ⚡ js/main.js          # JavaScript functionality
│
└── 📁 media/                  # User uploads
    ├── task_attachments/      # Task files
    └── avatars/               # Profile pictures
```

🔧 Advanced Setup

Custom Configuration

Create `.env` file in root directory:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
DATABASE_URL=sqlite:///db.sqlite3
ALLOWED_HOSTS=localhost,127.0.0.1
```

Production Deployment

For production deployment:

1. Set `DEBUG=False` in settings
2. Configure proper database (PostgreSQL recommended)

3. Set up static file serving
4. Configure email backend for notifications

Creating Custom Admin User

```
python manage.py createsuperuser
```

Making Existing User Admin

```
python manage.py shell
```

```
from django.contrib.auth.models import User
user = User.objects.get(username='your_username')
user.profile.role = 'admin'
user.profile.save()
exit()
```

Common Issues & Solutions

Issue: Template Not Found Error

Solution: Ensure template folder structure matches:

```
templates/
├── base.html
└── tasks/
    ├── task_list.html
    ├── task_detail.html
    └── task_confirm_delete.html
```

Issue: User Registered as Employee Instead of Admin

Solution: Use Django shell to change user role:

```
python manage.py shell
```

```
from django.contrib.auth.models import User
user = User.objects.get(username='your_username')
user.profile.role = 'admin'
user.profile.save()
```

Issue: Static Files Not Loading

Solution: Run collectstatic command:

```
python manage.py collectstatic
```

Technical Details

Built With:

- **Backend**: Django 4.2, Python 3.8+
- **Frontend**: Bootstrap 5.3, Font Awesome 6.4, jQuery 3.7
- **Database**: SQLite (development), PostgreSQL (production ready)
- **Authentication**: Django's built-in system with custom profiles

Key Models:

- **User & UserProfile**: Extended user system with roles
- **Task**: Core task management with approval workflow
- **Category & Priority**: Task organization and prioritization
- **Comment & TaskHistory**: Task collaboration and audit trail
- **Notification**: Real-time user notifications

Security Features:

- Role-based permissions
- CSRF protection
- Secure authentication
- File upload restrictions
- SQL injection protection

Customization

Adding New Task Categories

1. Login as admin
2. Go to Admin Panel → Categories
3. Add new categories with custom colors

Modifying User Roles

Edit `tasks/models.py` → `UserProfile.ROLE_CHOICES`

Customizing UI Colors

Edit `static/css/style.css` → `:root` CSS variables

Contributing

1. Fork the repository
2. Create feature branch (`git checkout -b feature/amazing-feature`)
3. Commit changes (`git commit -m 'Add amazing feature'`)
4. Push to branch (`git push origin feature/amazing-feature`)
5. Open Pull Request

License

This project is licensed under the MIT License - see the LICENSE file for details.

Support

Need help? Check these resources:

- **Documentation**: Review this README
- **Issues**: Check common issues section above
- **Django Docs**: https://docs.djangoproject.com/
- **Bootstrap Docs**: https://getbootstrap.com/docs/

What's Next?

After successful installation, you can:

1. **Explore the Admin Dashboard** - Login as admin to see management features
2. **Test the Workflow** - Create tasks and test the approval process
3. **Customize the System** - Add your own categories, priorities, and users
4. **Deploy to Production** - Follow production deployment guide above

Happy Task Managing!

Built with ❤ using Django and Bootstrap